The Latency-Energy Tradeoff of Surface-Code Computation: A Computed Frontier for the JPCUB Metric

Rowan Brad Quni-Gudzinas*

September 3, 2026

Abstract

Fault-tolerant quantum computation trades latency against energy through the code distance: a larger distance d costs more syndrome rounds and more physical gates per logical operation, lengthening every operation and raising its energy. This paper computes that tradeoff for the rotated surface code with a single explicit round-time convention. The model states its conventions up front: one logical operation takes t = d t_{round} (d syndrome rounds at round time t_{round}) and dissipates E = E_g 8 d^3 (the gate-energy convention of the companion energy model), so the latency-energy product scales as $d^{4\cdot}$ All quantitative claims are computed by a deposited verification script with t_{round} = 1 microsecond: the latency table from d = 3 (3.00 microseconds) to d = 31 (31.0 microseconds), the energy table from 2.160×10^{-10} J to 2.383×10^{-07} J, and the product table from 6.480×10^{-16} J s to 7.388×10^{-12} J s, spanning four orders of magnitude. The central result is that the product is dominated by the quartic scaling: doubling the code distance multiplies the latency-energy product by sixteen. The reader should care because the tradeoff is the cost function of fault-tolerant scheduling: when a target logical error rate fixes the required distance, it fixes both the latency and the energy of every logical operation, and the JPCUB metric - joules per logical-qubit operation - must be read together with the per-operation latency to price a workload. Where the premises end: the round time is a stated convention (1 microsecond, an order-of-magnitude surface-code round time); the gate count inherits the 8 d^3 convention; and the linear model neglects the parallel execution of independent logical operations, which trades wall-clock latency against energy in a way the single-operation analysis does not capture.

1 Introduction

A fault-tolerant computation has two costs per logical operation that are usually reported separately: how long it takes (latency) and how much energy it dissipates. Both grow with the code distance, but at different rates - latency linearly, energy cubically - and their product, the latency-energy cost of an operation, grows quartically. The tradeoff is the cost function that a scheduler or a roadmap must price.

This paper computes the tradeoff for the rotated surface code. With a single stated round-time convention, it produces the latency table, the energy table, and the product table across working distances, and it states the scaling law that organizes them: t ~ d, E ~ d^3, tE ~ $d^{4\cdot}$

The contribution is threefold: a convention-explicit statement of the two scaling laws (Section 3); a verified computation of the three tables (Section 5); and the reading of the JPCUB metric together with latency for workload pricing (Sections 4 and 7).

*doi:10.5281/zenodo.22281567 (version 2.0.0)

2 Prior work

The gate-counting and energy conventions follow the companion surface-code energy model and Fowler's overhead accounting [2]: d rounds of roughly 8 d^2 gates per round give N_g = 8 d^3, and the per-operation latency is d round times. The constant-overhead routes of Lavasani, Zhu, and Barkeshli [3] are the honest boundary: where the gate count per logical operation stops growing with distance, the cubic energy scaling and hence the quartic product are replaced by milder functions. The companion QNFO record on thermodynamic bottlenecks [4] motivates the energy-per-solution view within which the latency-energy product is the natural per-operation cost.

3 Model and conventions

Convention 1 (latency). One logical operation of a distance-d patch requires d syndrome rounds. With round time t_{round}, the per-operation latency is

t = d t_{round}.

Convention 2 (energy). The per-operation energy is

E = $E_g N_g$ = E_g 8 d^3,

with the gate-energy convention E_g = 1 pJ for the tables.

Convention 3 (round time). The tables use t_{round} = 1 microsecond, an order-of-magnitude surface-code round time. The product table scales linearly in t_{round}, so any measured round time substitutes directly.

Where the premises end: t_{round} and E_g are stated conventions, and the model prices a single logical operation in isolation; a real machine runs many logical operations in parallel, and wall-clock latency can be traded against energy by the degree of parallelism, which the single-operation analysis does not capture.

4 Analysis

The two scaling laws combine into one product law.

Latency grows linearly with distance: t = d t_{round}. Energy grows cubically: E = E_g 8 d^3. The product therefore grows quartically:

t E = d $t_{round} E_g$ 8 d^3 = 8 $t_{round} E_g d^4$.

The consequence is that the distance is the dominant lever on the product: doubling d multiplies tE by 16. Because the target logical error rate fixes the required distance (logical error scales as a power of the physical error raised to d/2), the error-correction requirement fixes both the latency and the energy of every logical operation before any scheduling decision is made.

The JPCUB metric is joules per logical operation; its product with the per-operation latency is the per-operation latency-energy cost. For workload pricing, the pair (JPCUB, t) is the complete per-operation cost in the single-operation model.

5 Results

All values in this section are computed by the deposited verification script ($jpcub_{nv}$_verify.py, section NV.005) with t_{round} = 1 microsecond and E_g = 1 pJ; no number is transcribed from memory.

Latency, energy, and product by distance.

The product spans four orders of magnitude across the table, matching the quartic law: the ratio of the d = 31 product to the d = 3 product is 1.14×10^4, close to $(31/3)^4 = 1.14 \times 10^4$.

Scaling check. The script verifies the quartic law directly: $(31/3)^4 = 1.140 \times 10^4$, and the table's product ratio reproduces it. Doubling d multiplies the product by 16.

d	t (s)	E (J)	tE (J s)
3	3.00×10^{-06}	2.160×10^{-10}	6.480×10^{-16}
5	5.00×10^{-06}	1.000×10^{-09}	5.000×10^{-15}
9	9.00×10^{-06}	5.832×10^{-09}	5.249×10^{-14}
15	1.50×10^{-05}	2.700×10^{-08}	4.050×10^{-13}
21	2.10×10^{-05}	7.409×10^{-08}	1.556×10^{-12}
31	3.10×10^{-05}	2.383×10^{-07}	7.388×10^{-12}

6 Discussion

Three consequences follow.

First, the distance is the single dominant decision. Because the product scales as d^4, every additional unit of code distance that the error budget demands is expensive in both time and energy, and the price accelerates. The error-correction requirement, not the controller, sets the frontier.

Second, the JPCUB metric alone is incomplete for workload pricing; it must be read with the per-operation latency. Two machines with the same joules per logical operation can have very different wall-clock performance if their round times differ, and the latency-energy product is the single number that captures both. The metric and its companion latency should be reported as a pair.

Third, the parallel-execution caveat is the main scope boundary. A real machine amortizes latency by running many logical operations concurrently, so the wall-clock cost of a workload is not simply the per-operation latency times the operation count. The per-operation product remains the correct unit of comparison; the degree of parallelism is a separate engineering variable that the single-operation model deliberately leaves out.

7 What a practitioner can do with this result

1. **Read JPCUB with latency.** Report joules per logical-qubit operation and the per-operation latency together. The product is the per-operation cost; reporting the energy alone hides the time dimension, and reporting the latency alone hides the energy.

1. **Price the distance before the schedule.** The target logical error rate fixes the required distance, and the distance fixes both latency and energy through t ~ d and E ~ $d^{3\cdot}$ Compute the required d first, then read the pair (t, E) from the tables; the product is the floor on the per-operation cost.

1. **Apply the scaling law.** Doubling the code distance multiplies the latency-energy product by sixteen. A fidelity improvement that saves one or two distance steps is worth a quartic factor in the product, which is the strongest single lever in the model.

1. **Substitute measured values.** The round time and gate energy are explicit conventions. Measured values replace t_{round} and E_g and the tables recompute in one line (the script encodes the substitution).

8 Conclusion

The latency-energy tradeoff of rotated surface-code computation is governed by two scaling laws and one product law: latency grows linearly with distance, energy cubically, and the product quartically. The computed tables place the tradeoff on a quantitative footing: from d = 3 to d = 31 the latency grows from 3.00 to

31.0 microseconds, the energy from 2.160×10^{-10} to 2.383×10^{-07} J, and the product from 6.480×10^{-16} to 7.388×10^{-12} J s, spanning four orders of magnitude with the quartic law verified. The JPCUB metric and the per-operation latency should be reported as a pair, and the distance, set by the error budget, is the dominant decision in the model. The deposited script reproduces every number.

References

[1] Landauer, R. 1961. "Irreversibility and Heat Generation in the Computing Process." IBM Journal of Research and Development 5 (3): 183-191.

[2] Fowler, A. G. 2012. "Low-overhead surface code logical Hadamard." arXiv:1202.2639.

[3] Lavasani, A., G. Zhu, and M. Barkeshli. 2019. "Universal logical gates with constant overhead: instantaneous Dehn twists for hyperbolic quantum codes." arXiv:1901.11029.

[4] QNFO. 2025. "Thermodynamic and Informational Bottlenecks of Scalable Fault-Tolerant Quantum Computation." Zenodo. doi:10.5281/zenodo.17955898.

Changelog

- v2.0 (2026-09-03): the product law tE ~ d^4 is derived and verified (product ratio 1.14×10^4 matches $(31/3)^{4)}$; the full latency, energy, and product tables are computed; prior-work positioning added (Sections 2 and 6); HTML and PDF renderings added to the deposit; abstract rewritten with computed values, no deferred claims.
- v1.0 (2026-09-03): initial short-form record.

Verification

Every number in Section 5 is produced by $jpcub_{nv}$_verify.py (section NV.005), deposited with this record, with t_{round} = 1 microsecond and E_g = 1 pJ as explicit conventions. The script computes the three tables and verifies the quartic law by comparing the product ratio across the table with $(31/3)^{4\cdot}$ Run "python $jpcub_{nv}$_verify.py" to reproduce the tables.